The EYES Manual

Zifei Zhong
(draft 2)

April 15, 2026

Contents

1 Design Purpose **3**

2 Data Characteristics of the Chronicle Log Files **3**

2.1 Data File Format 3

3 Design Philosophy **3**

4 Prerequisites **4**

5 Launching the Toolbox **5**

6 Execution Flow **5**

6.1 Phase A: Core Pre-processing 6

6.2 Phase B: Segment Analysis 6

6.2.1 Glance Logic 8

6.3 Phase C: Application & Event Logic 9

6.4 Phase D: Final Outputs 11

7 Usage Commands **12**

8 Log Events defined in the Android System **12**

1 Design Purpose

The EYES toolbox is designed to process and analyze log files collected from Android smart devices (phones and tablets) by the App Chronicle. The toolbox generates outputs such as App Usage, Device Pickups, Visualization, and integrated event timelines. The toolbox can be launched either from the command line or from a desktop GUI, where a user selects a folder containing raw Chronicle `.csv` files and runs the full pipeline with one click.

2 Data Characteristics of the Chronicle Log Files

The Chronicle log files are collected on a user device and then uploaded to a server. Some characteristics that we have observed in the data include:

1. Missing data: Some event logs might not be uploaded to the server successfully, thus causing missing data. For example, there may be a 'Screen Interactive' event in the log file, but the corresponding 'Screen Non-interactive' event is missing.
2. Duplication: Some event logs are uploaded multiple times, thus causing duplicated rows in the same file.
3. Out-of-order rows: In general, the log file records rows in increasing time order based on 'event_timestamp'. However, this is not always true. Many files contain rows recorded out of order.

There might be other issues that we are not aware of. We will keep the document updated when we have new observations.

2.1 Data File Format

The EYES toolbox takes raw data files as input. The raw data files are '.csv' files in which each row logs an event, including information such as 'app_package_name', 'interaction_type', and 'event_timestamp'. Figure 1 shows the raw data file format.

One recommended practice is to put all raw files in one directory and provide the path to this directory to the EYES toolbox. Then all raw data files in that directory will be processed.

3 Design Philosophy

The EYES toolbox analyzes the log data by:

4075_C01_AndroidTab Chronicle Android Raw Data 02-28-2025

study_id	participant_id	app_package_name	interaction_type	event_type	event_timestamp	timezone	username	application_label
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Configuration Change		2024-09-23T12:25:30.108-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Configuration Change		2024-09-23T12:25:51.807-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Configuration Change		2024-09-23T12:25:52.945-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.samsung.android.lool	Unknown importance: 10		2024-09-23T12:25:57.414-04:00	America/New_York		Device care
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.soundalive	Unknown importance: 11		2024-09-23T12:25:57.947-04:00	America/New_York		SoundAlive
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.samsung.tvmode	Unknown importance: 11		2024-09-23T12:25:58.099-04:00	America/New_York		TV Mode
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Configuration Change		2024-09-23T12:25:59.954-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Move to Background		2024-09-23T12:26:00.024-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Move to Foreground		2024-09-23T12:26:00.042-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Unknown importance: 23		2024-09-23T12:26:00.572-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Unknown importance: 16		2024-09-23T12:26:54.525-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Unknown importance: 15		2024-09-23T12:26:54.663-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	android	Unknown importance: 17		2024-09-23T12:26:54.706-04:00	America/New_York		Android System
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Move to Background		2024-09-23T12:26:54.956-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Unknown importance: 23		2024-09-23T12:26:55.008-04:00	America/New_York		Camera
358f0000-0000-0000-8000-000000000ade	4075_C01_AndroidTab	com.sec.android.app.camera	Move to Foreground		2024-09-23T12:26:56.791-04:00	America/New_York		Camera

Figure 1: Raw data file format.

- Identifying the internal states of the device: ACTIVE, GLANCE, IDLE, SHUTDOWN, GAP.
- Constructing the logic structure of an activity (triplet): Activity Resumed, Activity Paused, Activity Stopped.
- Inference: A set of inference rules is programmed into the toolbox scripts to ensure detailed analysis. For example, if no 'Activity Stopped' event exists, then the toolbox uses the pair ('Activity Resumed', 'Activity Paused') as the app usage interval; if only 'Activity Resumed' exists, then it uses the next app's 'Activity Resumed' event as the termination of the previous app usage interval.
- Combining the above to accurately extract App usage or device pickups.

Other data analysis tools also exist. For example, the Uzair (cite) scripts are widely used. However, the data processing logic of the Uzair tool is very different from that of the EYES toolbox. The Uzair tool simply matches each 'Activity Resumed' event to the next available 'Activity Paused' event and uses that interval as app usage. While this works for common app activity scenarios, it does not differentiate the activities of the same app well and it is not able to handle multi-window applications such as Youtube.

4 Prerequisites

The toolbox is developed in Python. Before running the EYES processing pipeline, ensure that you have:

1. Python 3 installed.
2. The EYES toolbox downloaded.
3. A terminal to run the execution commands (Unix/Linux terminal, or git bash on Windows), or access to the desktop GUI launcher.
4. A folder of raw Chronicle data files ready for processing.

5 Launching the Toolbox

The toolbox can be launched in two ways:

1. **GUI mode.** The GUI launcher allows a user to click `Browse`, select a folder containing raw Chronicle `.csv` files, and click `Go`. The GUI shows the current phase, elapsed run time, and command output in real time. This is the recommended mode for routine use by researchers who do not want to run multiple terminal commands manually.
2. **Command-line mode.** The toolbox can also be executed from a terminal by calling the `make` targets in the `Makefile`. This mode is convenient for debugging, scripting, or running only part of the pipeline.

6 Execution Flow

The pipeline follows a specific dependency chain. Each unit relies on the completion of the previous one. Figure 2 shows the dependency of the EYES processing units.

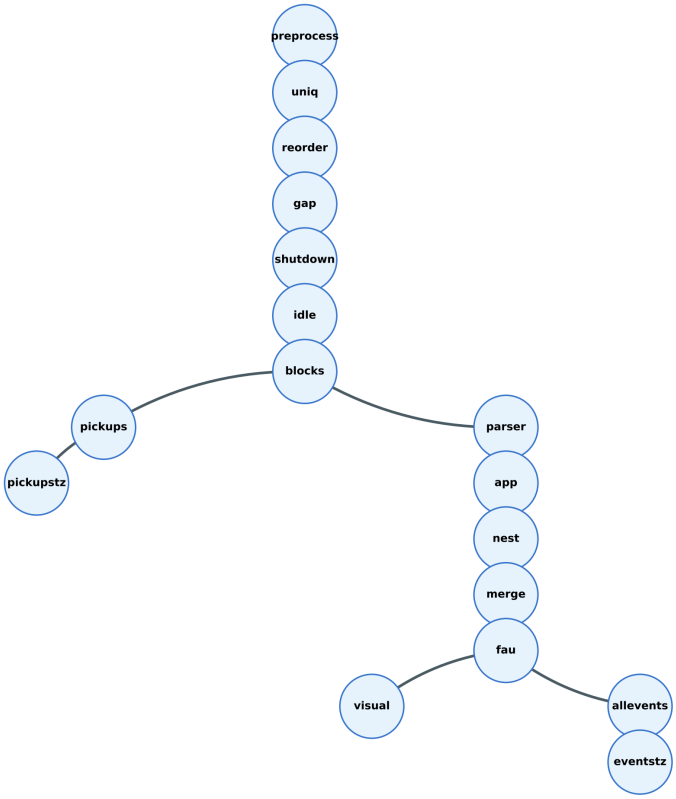


Figure 2: Dependency of the EYES Processing Units

6.1 Phase A: Core Pre-processing

There are three units in the pre-processing phase: `preprocess, uniq, reorder`.

1. `preprocess`: This unit does filtering and sorting.
   - In general, raw Chronicle files do not need to be filtered. However, if a user wants to filter out certain rows based on 'interaction type', the scripts provide this functionality. For example, raw files often contain thousands of rows with 'Unknown importance: 11'. Filtering out these rows can reduce data size and speed up processing.
   - Sorting is necessary. Since rows in the log file are not always recorded in time-increasing order, we sort the rows by 'event timestamp' to maintain a time-increasing order.
2. `uniq`: Removes duplicate entries.

   Once the data rows are sorted in the previous unit, `preprocess`, this unit, `uniq`, removes duplicated rows from the data file.
3. `reorder`: Reorder two particular events (Activity Resumed, Activity Paused) when they have identical timestamps.

   This is needed because identical timestamps create ambiguity: according to the timestamp, the two events happen at the same time, but logically one must occur before the other. The `reorder` unit is designed to place the 'Activity Paused' row before the 'Activity Resumed' row if both rows have identical timestamps. This resolves the ambiguity. If 'Activity Paused' is in fact behind 'Activity Resumed' (and thus creates a 0-duration interval between the two events), our `parser` unit is designed to catch it even if the rows are presented in the wrong order.

   For log files collected from old versions of the Android system, we match the tag 'Move to Foreground' and 'Move to Background' to 'Activity Resumed' and 'Activity Paused', respectively.

6.2 Phase B: Segment Analysis

The processing units in this phase identify explicit non-active device states and then derive ACTIVE as the remaining time. The states are: ACTIVE, GLANCE, GAP, SHUTDOWN, and IDLE. The SHUTDOWN state indicates that a device is powered off. The IDLE state indicates that a device is in an idle mode in which the screen is off or the device is otherwise not interactive. The GAP state indicates that a device is in either SHUTDOWN or IDLE, but the toolbox cannot determine which state is more likely based on the available data. The GLANCE state indicates that the device screen briefly lights up but does not become unlocked before it turns off again.

In the current design, ACTIVE does not mean that the user is definitely performing meaningful activity. Rather, ACTIVE is the complement of the explicit non-active states:

$$\texttt{ACTIVE} = \neg(\texttt{SHUTDOWN} \cup \texttt{IDLE} \cup \texttt{GAP} \cup \texttt{GLANCE})$$

We later combine ACTIVE periods with app activity results to determine whether real user activity occurred.

If a GAP state is present, the user should inspect the logs to determine the likely device state, because in this case the EYES toolbox cannot determine confidently whether the state is SHUTDOWN or IDLE. In general, the user should be alerted if the processing results include too many GAP states.

1. `gap`: Identifies missing-data periods. This step scans the data rows and compares the timestamps of two consecutive rows. If the timestamp difference is greater than a specified gap threshold, a gap is identified. The gap threshold is configurable, and its default value is 3 hours. If a gap has a duration shorter than 3 hours, it will not be detected by default. A good practice is to process the log data multiple times with different parameters and compare the results to choose the best one.

2. `shutdown`: Detects the periods when the device was powered off. This is done by analyzing the events related to device shutdown/startup in the log file.

3. `idle`: Identifies periods of inactivity. This is done by analyzing the events related to screen interactive/non-interactive and keyguard events in the log file.

4. `glance`: Identifies wake-up periods where the device screen becomes interactive but the device is not unlocked before the screen turns off again. This state is useful because otherwise such periods would be counted as ACTIVE by complement, even though they often do not correspond to real user usage.

5. `blocks`: Categorizes segments and merges them into a single list of blocks in time-increasing order based on the start time of each block.

6. `pickups`: Generates the *pickups* data for the device. The *pickups* data indicates how many times the device was picked up during the observation period. Figure 3 shows the generated pickups data. Rows labeled 'ACTIVE' indicate that the device is being used during the interval from 'start_ts' to 'end_ts'.

4755_P01_Android_Chronicle_Android_Raw_Data_06-05-2025_sorted_uniq_reordered_PIC

start_ts	end_ts	duration_seconds	block_type
2025-03-30 15:18:04.781000+00:00	2025-03-30 15:23:09.264000+00:00	304	IDLE
2025-03-30 15:23:09.264000+00:00	2025-03-30 15:23:34.490000+00:00	25	ACTIVE
2025-03-30 15:23:34.490000+00:00	2025-03-30 15:25:54.231000+00:00	139	IDLE
2025-03-30 15:25:54.231000+00:00	2025-03-30 15:27:50.033000+00:00	115	ACTIVE
2025-03-30 15:27:50.033000+00:00	2025-03-30 15:37:32.878000+00:00	582	IDLE
2025-03-30 15:37:32.878000+00:00	2025-03-30 15:38:02.688000+00:00	29	ACTIVE
2025-03-30 15:38:02.688000+00:00	2025-03-30 15:39:00.358000+00:00	57	IDLE
2025-03-30 15:39:00.358000+00:00	2025-03-30 15:40:00.362000+00:00	60	ACTIVE

Figure 3: EYES processing result: Pickups.

For each identified period, there is an associated 'start time', 'end time', and 'type'. We call such a period a **segment**. For a specific segment type, segments are linked together in time-increasing order by their 'start time'. Note that a 'gap' segment might overlap with a 'shutdown' segment.

In the `blocks` step, we use a more sophisticated algorithm to analyze these segments collectively and identify the state of the device for each period. The outcome of this step is a list of blocks linked in time-increasing order based on the start time of each block. This list captures the device's state changes over time.

6.2.1 Glance Logic

The `GLANCE` state is constructed from the same family of system events used for IDLE analysis, especially:

- `SCREEN_INTERACTIVE` (Unknown importance: 15)
- `SCREEN_NON_INTERACTIVE` (Unknown importance: 16)
- `KEYGUARD_HIDDEN` (Unknown importance: 18)

The logic is:

1. When a `Screen Interactive` event occurs at time t_1, the toolbox starts a candidate wake episode.
2. The candidate episode is examined until the first decisive event after t_1:
   - `Keyguard Hidden`
   - `Screen Non-interactive`
   - a SHUTDOWN boundary
   - a GAP boundary
3. If the first decisive event is `Keyguard Hidden`, then the device is treated as unlocked and the interval is **not** labeled as GLANCE.
4. If `Screen Non-interactive` occurs at time t_2 and no `Keyguard Hidden` event occurs during the interval $[t_1, t_2]$, then the interval is labeled as `GLANCE`.
5. If a SHUTDOWN or GAP boundary happens before the screen returns to non-interactive, the candidate wake interval is discarded from GLANCE detection because the wake episode did not close normally.

Therefore, GLANCE captures the behavioral scenario: the screen lights up, but the user never unlocks the device before the screen turns off again.

There is an alternative, more conservative interpretation that could be used in future work. In that alternative design, if a `Keyguard Hidden` event occurs very shortly *after* `Screen Non-interactive`, the earlier wake interval might be treated as not meaningful enough to retain as a separate glance. However, this is **not** the current implementation in the EYES toolbox.

The current implementation only checks whether `Keyguard Hidden` occurs *within* the candidate wake interval. If the screen becomes non-interactive first, the interval is retained as `GLANCE`, even if a later unlock-related event is recorded shortly afterward. We keep this implementation for now because it preserves a simple and reproducible interval rule, and because such a glance immediately preceding a later ACTIVE period is not expected to materially affect downstream ACTIVE usage analysis.

6.3 Phase C: Application & Event Logic

1. `parser`: Parses app activity events to calculate activity-level app usage. The parser respects the logic structure of an app activity. The structure consists of three events: *Activity Resumed*, *Activity Paused*, and *Activity Stopped*.

   In general, an app might have multiple activities. For example, when a user interacts with an app that has multiple panels, each of which may correspond to a unique activity, an *Activity Resumed* event will be triggered when the user navigates to a new panel.

   If a user launches an App and stays on the first panel A (Activity A), then navigates to a new panel B (Activity B), multiple events will be triggered in sequence, as illustrated in Figure 4. Notice that the three events – Activity Paused (A), Activity Resumed (B), and Activity Stopped (A) – happen at almost the same time (usually within 1 or 2 seconds). We consider the time duration from Activity Resumed (A) to Activity Stopped (A) as the actual usage time for activity A.

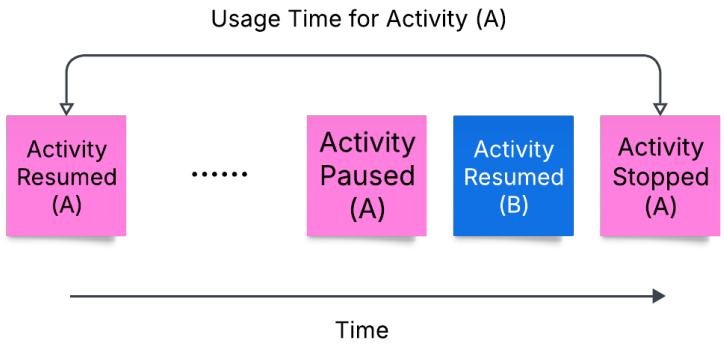


Figure 4: Android Device Event Ordering for Activity A and Activity B.

2. `app`: Calculates app-level usage time.

   Based on the activity-level usage data processed by the `parser`, in this step we merge the activity usage of the same app to produce app-level usage data. The processed data show all apps and their usage durations, sorted by the resumed time (*Activity Resumed*) of each app.

4075_C01_app_1

app_package_name	resume_time	end_time	duration	app_label
com.sec.android.app.camera	2024-09-23 16:26:00.042000+00:00	2024-09-23 16:26:55.008000+00:00	54.966	Camera
com.sec.android.app.camera	2024-09-23 16:26:56.791000+00:00	2024-09-23 16:27:17.925000+00:00	21.134	Camera
com.sec.android.app.camera	2024-09-23 16:27:18.078000+00:00	2024-09-23 16:27:20.238000+00:00	2.16	Camera
com.sec.android.app.camera	2024-09-23 16:27:37.909000+00:00	2024-09-23 16:27:51.912000+00:00	14.003	Camera
com.sec.android.app.camera	2024-09-23 16:28:09.745000+00:00	2024-09-23 16:30:11.115000+00:00	121.37	Camera
com.sec.android.app.camera	2024-09-23 16:30:18.051000+00:00	2024-09-23 16:30:38.483000+00:00	20.432	Camera
com.sec.android.app.camera	2024-09-23 16:30:41.175000+00:00	2024-09-23 16:30:41.408000+00:00	0.233	Camera

Figure 5: EYES processing result: App Level Usage Data.

3. `nest`: Performs secondary nested analysis on App usage.

   There are scenarios in which a multi-window app (for example, Youtube) is running while another app is also being used by the user. In this case, both apps may have significant overlapping usage time. We still collect the app-level usage and later identify the overlap so that we do not double count usage time. For a given overlapping interval, the processing result will show that one app is the primary usage and the other app is the secondary usage.

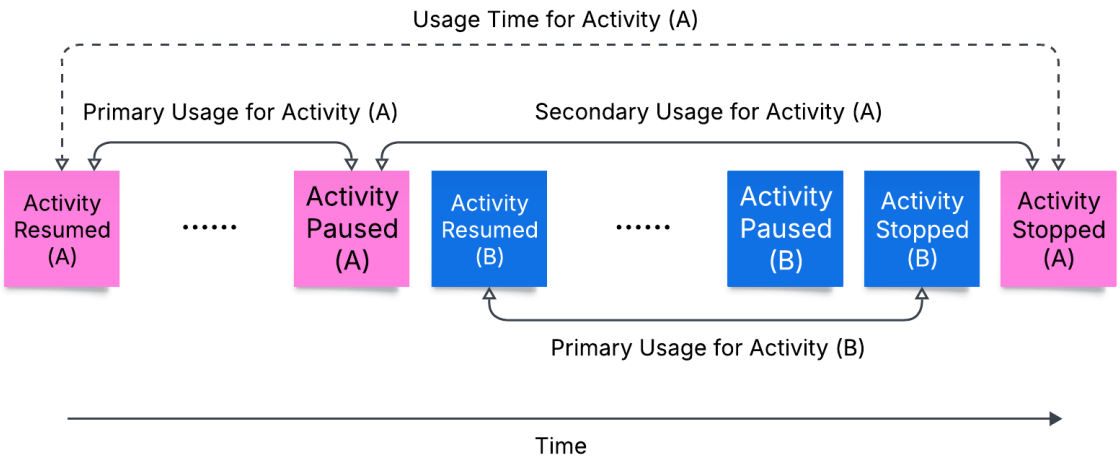


Figure 6: Primary Usage vs. Secondary Usage

   Figure 7 shows the data generated by the `nest` unit. There is a 'secondary_duration' column when a given duration is classified as secondary usage for an app.

4. `merge`: Merges device status data with app usage data. This merged dataset has one extra column that labels the device status for each row of app usage data. This extra information is very important for interpreting the usage of a given app. For example, if app usage occurs while the device is in IDLE state, that may indicate that the app was running but the user was not really using it. Figure 8 shows the file format of the merged data.

5. `fau`: Calculates "Final App Usage" (FAU) time, or effective screen time. Once we have the data processed by the `merge` unit, we use a simple approach to sum all app usage time for a given app under the condition that the device status is 'ACTIVE' during the app-usage

4075_C01_primary_secondary_nested

app_package_name	resume_time	end_time	duration	secondary_duration	app_label
com.sec.android.app.camera	2024-09-23 16:26:00.042000+00:00	2024-09-23 16:26:55.008000+00:00	54.966	0	Camera
com.sec.android.app.camera	2024-09-23 16:30:41.175000+00:00	2024-09-23 16:30:41.408000+00:00	0.233	0	Camera
com.sec.android.gallery3d	2024-09-23 16:30:38.193000+00:00	2024-09-23 16:30:41.175000+00:00	2.982	0	Gallery
com.sec.android.gallery3d	2024-09-23 16:30:41.175000+00:00	2024-09-23 16:30:41.408000+00:00	0	0.233	Gallery
android	2024-09-23 16:30:37.902000+00:00	2024-09-23 16:30:38.193000+00:00	0.291	0	Android System
android	2024-09-23 16:30:38.193000+00:00	2024-09-23 16:30:38.256000+00:00	0	0.063	Android System
com.sec.android.app.camera	2024-09-23 16:30:18.051000+00:00	2024-09-23 16:30:37.902000+00:00	19.851	0	Camera
com.sec.android.app.camera	2024-09-23 16:30:37.902000+00:00	2024-09-23 16:30:38.483000+00:00	0	0.581	Camera
com.sec.android.app.launcher	2024-09-23 16:30:15.969000+00:00	2024-09-23 16:30:18.051000+00:00	2.082	0	One UI Home

Figure 7: EYES processing result: Primary and Secondary Usage Data.

4075_C01_primary_secondary_fau

app_package_name	resume_time	end_time	duration	secondary_duration	app_label	device_status
com.sec.android.app.camera	2024-09-23 16:26:00.042000+00:00	2024-09-23 16:26:55.008000+00:00	54.966	0.0	Camera	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:17.823000+00:00	2024-09-23 16:27:18.078000+00:00	0.255	0.0	One UI Home	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:18.078000+00:00	2024-09-23 16:27:18.328000+00:00	0.0	0.25	One UI Home	ACTIVE
com.sec.android.app.camera	2024-09-23 16:27:18.078000+00:00	2024-09-23 16:27:19.744000+00:00	1.666	0.0	Camera	ACTIVE
com.sec.android.app.camera	2024-09-23 16:27:19.744000+00:00	2024-09-23 16:27:20.238000+00:00	0.0	0.494	Camera	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:19.744000+00:00	2024-09-23 16:27:37.909000+00:00	18.165	0.0	One UI Home	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:37.909000+00:00	2024-09-23 16:27:38.556000+00:00	0.0	0.647	One UI Home	ACTIVE
com.sec.android.app.camera	2024-09-23 16:27:37.909000+00:00	2024-09-23 16:27:51.772000+00:00	13.863	0.0	Camera	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:51.772000+00:00	2024-09-23 16:27:58.678000+00:00	6.906	0.0	One UI Home	ACTIVE
com.sec.android.app.camera	2024-09-23 16:27:51.772000+00:00	2024-09-23 16:27:51.912000+00:00	0.0	0.14	Camera	ACTIVE
com.samsung.android.app.galaxyfinder	2024-09-23 16:27:58.678000+00:00	2024-09-23 16:28:09.745000+00:00	11.067	0.0	Finder	ACTIVE
com.sec.android.app.launcher	2024-09-23 16:27:58.678000+00:00	2024-09-23 16:28:00.286000+00:00	0.0	1.608	One UI Home	ACTIVE
com.sec.android.app.camera	2024-09-23 16:28:09.745000+00:00	2024-09-23 16:30:09.509000+00:00	119.764	0.0	Camera	ACTIVE

Figure 8: EYES processing result: App Usage and Device Status.

interval. We refer to this as the FAU time for an app.

If for some reason a researcher wants to sum an app's usage under device status 'ACTIVE' or 'IDLE', it should be easy to do using the results from the `merge` unit.

6.4 Phase D: Final Outputs

1. `visual`: Generates a visual display of app usage for a given raw file. This helps researchers understand the data more easily. Figure 9 shows an example of app-usage visualization. From the figure, it is easy to get an overview of app usage for each day.

2. `allevents`: Compiles a comprehensive list of all detected events and app usage. This output file includes 1) all activity events, 2) all app usage, and 3) device status.

3. `eventstz`: This unit adds timezone information to the data generated by `allevents`. This is the final unit specified in the Makefile. We can use the commands from Table 1 to run this unit, and it will trigger all earlier units sequentially. Figure 10 shows the format of the data that integrates all activity events, app usage, and device status.

7 Usage Commands

Table 1 lists the commands that a researcher can use to run the EYES toolbox scripts. All commands require the data directory to be specified using the `dir` variable.

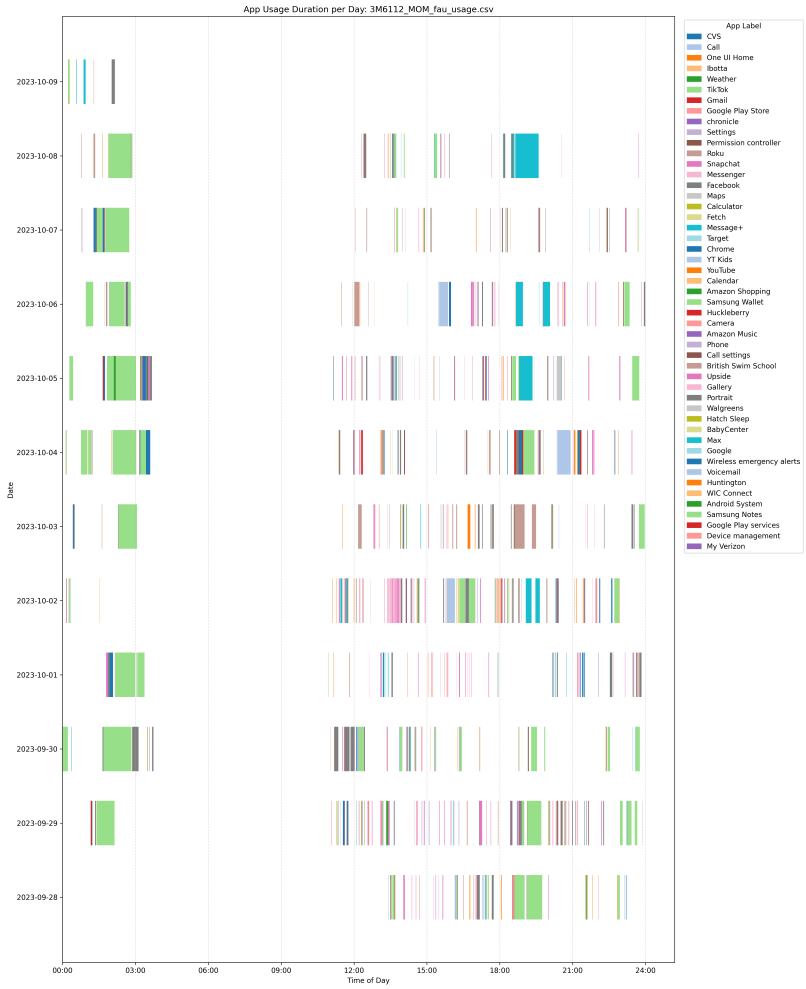


Figure 9: EYES processing result: Visualization of App Usage.

8 Log Events defined in the Android System

Table 2 shows all the events that are defined in the Android System (in the file `UsageEvents.java`):

4075_P01_events_tz

app_package_name	resume_time	end_time	duration	secondary_duration	app_label	device_status	interaction_type	timezone	tz_offset	local_start_time	local_end_time
android	2024-09-23 15:53:34.731+00:00				Android System		Unknown importance: 15	America/New_York	-04:00	2024-09-23 11:53:34.731000-04:00	
com.reddit.frontpage	2024-09-23 15:53:36.258+00:00	2024-09-23 15:55:56.566+00:00	140.308	0.0	Reddit	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:53:36.258000-04:00	2024-09-23 11:55:56.566000-04:00
android	2024-09-23 15:53:36.320+00:00				Android System		Unknown importance: 18	America/New_York	-04:00	2024-09-23 11:53:36.320000-04:00	
com.facebook.orca	2024-09-23 15:55:05.928+00:00				Messenger		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:55:05.928000-04:00	
com.facebook.orca	2024-09-23 15:55:08.519+00:00				Messenger		Unknown importance: 10	America/New_York	-04:00	2024-09-23 11:55:08.519000-04:00	
com.facebook.orca	2024-09-23 15:55:11.657+00:00				Messenger		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:55:11.657000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:51.996+00:00				Messages		Unknown importance: 19	America/New_York	-04:00	2024-09-23 11:55:51.996000-04:00	
com.google.android.apps.tachyon	2024-09-23 15:55:52.906+00:00				Meet		Unknown importance: 11	America/New_York	-04:00	2024-09-23 11:55:52.906000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:53.045+00:00				Messages		Unknown importance: 20	America/New_York	-04:00	2024-09-23 11:55:53.045000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:53.245+00:00				Messages		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:55:53.245000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:55.330+00:00				Messages		Unknown importance: 19	America/New_York	-04:00	2024-09-23 11:55:55.330000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:55.915+00:00				Messages		Unknown importance: 20	America/New_York	-04:00	2024-09-23 11:55:55.915000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:56.116+00:00				Messages		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:55:56.116000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:56.346+00:00				Messages		Unknown importance: 10	America/New_York	-04:00	2024-09-23 11:55:56.346000-04:00	
com.facebook.orca	2024-09-23 15:55:56.347+00:00				Messenger		Unknown importance: 10	America/New_York	-04:00	2024-09-23 11:55:56.347000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:56.549+00:00				Messages		User Interaction	America/New_York	-04:00	2024-09-23 11:55:56.549000-04:00	
com.samsung.android.messaging	2024-09-23 15:55:56.566+00:00	2024-09-23 15:56:03.757+00:00	7.191	0.0	Messages	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:55:56.566000-04:00	2024-09-23 11:56:03.757000-04:00
com.reddit.frontpage	2024-09-23 15:55:56.566+00:00	2024-09-23 15:55:57.594+00:00	0.0	1.028	Reddit	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:55:56.566000-04:00	2024-09-23 11:55:57.594000-04:00
com.samsung.android.messaging	2024-09-23 15:56:03.757+00:00	2024-09-23 15:56:04.414+00:00	0.0	0.657	Messages	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:56:03.757000-04:00	2024-09-23 11:56:04.414000-04:00
com.opera.browser	2024-09-23 15:56:03.757+00:00	2024-09-23 15:56:27.755+00:00	23.998	0.0	Opera	ACTIVE	App Usage	America/New_York	-04:00	2024-09-23 11:56:03.757000-04:00	2024-09-23 11:56:27.755000-04:00
com.samsung.android.app.smartcapture	2024-09-23 15:56:25.558+00:00				Samsung capture		Unknown importance: 12	America/New_York	-04:00	2024-09-23 11:56:25.558000-04:00	
com.sec.android.gallery3d	2024-09-23 15:56:27.754+00:00				Gallery		Unknown importance: 11	America/New_York	-04:00	2024-09-23 11:56:27.754000-04:00	

Figure 10: EYES processing result: All events, App Usage, and Device Status.

Table 1: Major commands to run the EYES toolbox scripts

Full Process	`make all dir=/path/to/data`	Run the entire pipeline (Visuals + Pickups)
GUI Launcher	`make gui`	Launch the EYES desktop GUI. Select a raw-data folder and click `Go` to run the pipeline.
Clean Up	`make clean dir=/path/to/data`	Delete all the processed results (to start over)
Specific Unit	`make [target] dir=/path/to/data`	Run up to a specific target (e.g., `make app`)
The final unit	`make eventstz dir=/path/to/data`	Run timezone column adjustment. This will also run all the units ahead sequentially.

Table 2: Andorid System Events and Numbering

Event Name	Unknown importance #
ACTIVITY_RESUMED	1
ACTIVITY_PAUSED	2
END_OF_DAY	3
CONTINUE_PREVIOUS_DAY	4
CONFIGURATION_CHANGE	5
SYSTEM_INTERACTION	6
USER_INTERACTION	7
SHORTCUT_INVOCATION	8
CHOOSER_ACTION	9
NOTIFICATION_SEEN	10
STANDBY_BUCKET_CHANGED	11
NOTIFICATION_INTERRUPTION	12
SLICE_PINNED_PRIV	13
SLICE_PINNED	14
SCREEN_INTERACTIVE	15
SCREEN_NON_INTERACTIVE	16
KEYGUARD_SHOWN	17
KEYGUARD_HIDDEN	18
FOREGROUND_SERVICE_START	19
FOREGROUND_SERVICE_STOP	20
CONTINUING_FOREGROUND_SERVICE	21
ROLLOVER_FOREGROUND_SERVICE	22
ACTIVITY_STOPPED	23
ACTIVITY_DESTROYED	24
FLUSH_TO_DISK	25
DEVICE_SHUTDOWN	26
DEVICE_STARTUP	27
USER_UNLOCKED	28
USER_STOPPED	29
LOCUS_ID_SET	30
APP_COMPONENT_USED	31